講稿規劃

兩個版本都是「講完整份簡報 p1~p14，並搭配現場實作」: 6 分鐘版、10 分鐘版。

一、先看時間帳

實作步驟	實際耗時	來源
① 注入故障 → ③ 收到告警信	約 2 分鐘	p10 實測
② `curl` 取得 500	每次約 60 秒 (別中斷)	README 故障演練注意事項
④b AI 調查完成	約 10 分鐘 (從 Alarm 觸發起算)	你的實測
⑤ `terraform apply` 還原	很快 (只改 SG 規則，EC2、RDS 都不變動)	你的實測

結論: 從注入故障到 AI 報告產出約 12 分鐘，比 6 分鐘版、10 分鐘版都長。 所以 AI 報告不可能在台上現場等到，必須事先處理，不能靠「等」。

二、整體策略: 邊講邊跑，報告用預跑結果

1. **開場就注入故障**，之後的等待時間用 p3~p8 的非實作內容填滿。你講情境、痛點、解法、成效、選型的同時，故障、告警都在背景跑，不需要乾等。
2. **告警信剛好在 p6 前後到**，p6 本來就在講「1~2 分鐘告警信」，現場確認信箱就是最好的佐證，不是額外負擔。
3. **AI 報告用預跑結果**，不現場等。p11 打開事先跑好的調查頁面 (或簡報上的 screenshot8)，口頭坦白說明:「調查約需 10 分鐘，這是我先前跑完的結果，今天這一輪還在跑」。
4. **今天這一輪的報告，留到 Q&A 揭曉**。注入起算約 12 分鐘才會跑完，6 分鐘版、10 分鐘版剛好都落在 Q&A 期間，被問到再打開，當作加分。
5. `terraform apply` **很快，不需要留等待時間**: 在 p11 看完報告後直接下指令，看輸出、curl 確認 200，一氣呵成。
6. 備援: 任何一步現場沒如期發生 (告警沒到、curl 卡住)，直接切換到簡報上對應的

screenshot，口頭說明「先看之前跑的結果」，不要停下來乾等。

三、現場操作與視窗配置 (事先排好)

視窗	內容	用途
終端機 1	`infra/` 目錄，預先貼好故障注入指令 (README 步驟 ①)	注入故障、`terraform apply` 還原
終端機 2	預先貼好 `curl` 指令 (README 步驟 ②)，建議包成迴圈連續送 2~3 次	持續製造 5xx，同時展示 500
瀏覽器分頁 1	CloudWatch Alarms 的 `alarm-yiweee-http-5xx`	步驟 ③
瀏覽器分頁 2	信箱 (AWS Notifications)	步驟 ④a
瀏覽器分頁 3	CloudWatch investigations: **預跑的那筆調查** (Hypotheses 段落)	步驟 ④b，報告來源
瀏覽器分頁 4	今天這輪 investigations 頁面	Q&A 揭曉
簡報	全螢幕	主畫面

演練前確認 (README):

- 網站回 200，Alarm 狀態為 `INSUFFICIENT_DATA`。
- 預跑的調查頁面備好 (能否使用等彩排後再定；不行就用簡報 screenshot8)。
- 開講時記下注入的時刻 (現場時間)，p10 要念。簡報上的 13:22 / 13:24 是先前那次的數字，現場念現場的時間，或念「約 2 分鐘」。

四、時間軸總表

「T」是講稿累計時間，▸ 是操作動作，不佔獨立時間，都包含在該頁時間內。

6 分鐘版 (預估 5 分 30 秒，緩衝 30 秒)

頁	時間	累計	▸ 現場操作	背景進度
p1~p2	0:25	0:25	▸ 結尾切終端機 1 注入故障、切終端機 2 開始 curl	故障起點
p3	0:25	0:50	—	curl 等待中
p4	0:15	1:05	—	curl 等待中
p5	0:35	1:40	—	curl 回 500
p6	0:30	2:10	—	Alarm 即將觸發
p7~p8	0:25	2:35	—	告警信寄出 (約 T=2:25)
橋接	0:10	2:45	▸ 切分頁 1、2: 確認 Alarm 與告警信	—
p9	0:30	3:15	—	調查跑中
p10	0:45	4:00	▸ 展示終端機 2 的 500、分頁 1、分頁 2	調查跑中
p11	0:50	4:50	▸ 看分頁 3 的報告；再到終端機 1 下 `terraform apply`；最後 curl 200	調查仍在跑
p12	0:20	5:10	—	—
p13~p14	0:20	5:30	—	今天這輪報告約 T=12:30 完成

10 分鐘版 (預估 9 分 25 秒，緩衝 35 秒)

頁	時間	累計	▸ 現場操作	背景進度
p1~p2	0:30	0:30	▸ 結尾切終端機 1 注入故障、切終端機 2 開始 curl	故障起點
p3	0:45	1:15	—	curl 等待中
p4	0:25	1:40	—	curl 回 500

頁	時間	累計	▸ 現場操作	背景進度
p5	0:55	2:35	—	告警信約 T=2:30 寄出
p6	0:50	3:25	▸ 講到「1~2 分鐘告警信」時切分頁 1、2 現場確認	調查跑中
p7	0:45	4:10	—	—
p8	0:35	4:45	—	—
p9	0:55	5:40	—	—
p10	1:10	6:50	▸ 展示終端機 2 的 500、分頁 1、分頁 2	調查仍在跑
p11	1:20	8:10	▸ 看分頁 3 的報告；再到終端機 1 下 `terraform apply`；最後 curl 200	調查仍在跑
p12	0:40	8:50	—	—
p13~p14	0:35	9:25	—	今天這輪報告約 T=12:30 完成

兩個版本的差別: 前半多講選型推導與用戶規模；p6 的現場確認放在 p6 內 (10 分鐘版告警剛好在這時到)，6 分鐘版則用橋接段，避免 p6 時告警還沒到。

五、6 分鐘講稿

【】內為操作或提示，不念出來。

[p1] 封面 + [p2] 目次 (0:25)

大家好，我是 14 號王奕惟。今天分享「Taiwan Sip」，一個把台灣手搖飲賣到日本的 WordPress 網路商店。這次簡報我會邊講邊做實作，所以我先把故障注入，接下來你們會看到系統怎麼反應。
【切終端機 1 注入故障，記下時間；切終端機 2 開始 curl；回到簡報】

[p3] 情境與目標 (0:25)

這是正常運作的首頁。2025 年訪台日本旅客 150 萬人次，約 0.3% 成為平台用戶，就是 4,500

人，每人一年上來 4 次，每天約 50 人次。目標有兩個: 資料庫不對外，故障時自動產出根因報告。

[p4] 兩個痛點 (0:15)

背後是兩個痛點: 平台涉及用戶隱私，資料庫要安全；公司規模小，維運人力不足。

[p5] 解法 (0:35)

安全靠內外網隔離: 只有 Web 能連 DB，Internet 無法直連。人力靠 AI 自動調查: Web 日誌自動送到監控平台，AI 調查日誌與 AWS 操作紀錄，直接產出根因報告。前面全自動，維運人員只需要讀報告。

[p6] 成效 (0:30)

安全上，導入前 DB 暴露在外網，導入後只在內網。人力上，故障後 1~2 分鐘收到告警信；以前要登機器翻 log、查 RDS 指標、查 AWS 操作紀錄、找到根因，共四個人工步驟，現在變成一個 AI 自動步驟。這些我稍後用實測證明。

[p7~p8] 選型 (0:25)

選型只講結論。EC2 選 t3.micro，因為看記憶體不看 CPU: CPU 只用到 1%，記憶體需求 760MB，t3.nano 的 0.5GiB 不夠，t3.micro 的 1GiB 才夠。資料庫選 RDS，每月 21.74 美元，比自建 MySQL 的 9.93 美元多約 12 美元，換來備份、修補、還原由 AWS 處理。

[橋接] (0:10)

我們回頭看開場注入的故障。
【切分頁 1、2: Alarm 是否為 ALARM，告警信是否到達】
距離我注入已經過了約兩分鐘，Alarm 已經觸發，告警信也到了。
【若尚未到: 「還沒到，p10 再看」，繼續講，不要停】

[p9] 實作 - 架構 (0:30)

這是架構。ALB 在 public subnet 接 HTTPS 443，EC2 跑 WordPress，RDS 在 private subnet，只開 SG 3306 給 Web，Internet access gateway 是 Disabled。EC2 上的 CloudWatch Agent 把 Apache log 推到 CloudWatch，轉成 5xx 指標，Alarm 觸發後平行寄告警信，並啟動 investigations 自動調查。

[p10] 故障演練 (1/2) (0:45)

剛才的故障就是這六步的前三步。第一步移除 DB 安全群組的 3306 規則。
【切終端機 2】第二步，網站回傳 500，這是剛剛現場的結果。
【切分頁 1、2】第三步 Alarm 觸發，這是告警信。現場從注入到收到告警信約 2 分鐘，印證 p6 的「1~2 分鐘」。
【若現場沒到: 看簡報上的 screenshot6、7，說「先看之前跑的結果」】

[p11] 故障演練 (2/2) (0:50)

Alarm 觸發後 investigations 自動調查，約需 10 分鐘，所以今天這輪的報告還在跑。
【切分頁 3】這是我先前跑完的結果: Hypotheses 明確指出 RevokeSecurityGroupIngress 事件，也就是 SG 規則被移除。
有了根因，我直接還原。
【切終端機 1 下 terraform apply，輸出看到 SG rule 被加回；再切終端機 2，curl 回 200】
網站恢復正常。人工排查從 4 步變成 1 步: 不用登機器翻 log，報告直接指出根因。

[p12] 權限安全 (0:20)

讓 AI 查系統，會不會多開一個洞？AI 能查，不能改。investigations role 只能讀 Logs、RDS 指標、CloudTrail，政策全是 Get、List、Describe；全部用 IAM role，沒有長期金鑰。

[p13] Takeaway + [p14] 結尾 (0:20)

總結，這是安全、省人力的 WordPress 平台: 內外網隔離，DB 只在內網；AI 自動調查，約 2 分鐘告警，人工排查 4 步變 1 步。今天這輪的 AI 報告再過幾分鐘會完成，Q&A 時我可以現場展示。謝謝大家。

六、10 分鐘講稿

[p1] 封面 + [p2] 目次 (0:30)

大家好，我是 14 號王奕惟。今天分享「Taiwan Sip 啜飲台灣手搖飲料網路商店」，主題是用 Web/DB 分離與 CloudWatch 自動調查，打造安全、省人力的 WordPress 平台。內容依序是情境、痛點、解法、成效、選型，最後是實作。因為故障偵測和 AI 調查都要等，我先把故障注入，邊講邊讓系統跑。
【切終端機 1 注入故障，記下時間；切終端機 2 開始 curl；回到簡報】

[p3] 情境與目標 (0:45)

這是平台正常運作的首頁，我們把台灣手搖飲即時包賣給日本消費者。規模怎麼估: 2025 年訪台日本旅客 150 萬人次，假設約 0.3% 成為平台用戶，就是 4,500 人；每人一年上平台約 4 次，每天約 50 人次。規模不大，所以後面選型會很節制。目標有兩個: 資料庫不對外，故障時自動產出根因報告。

[p4] 兩個痛點 (0:25)

第一，資料庫安全性: 平台涉及用戶隱私，外洩代價高。第二，維運人力不足: 公司規模小，沒有專職維運，必須減輕負擔。

[p5] 解法 (0:55)

第一，內外網隔離: 採網段隔離，只有 Web 平台能連線資料庫，Internet 無法直連 DB。第二，AI 自動調查: Web 日誌自動拋轉到監控平台，由監控平台的 AI 調查 Web 日誌、AWS 資源操作紀錄等，提供根因分析報告。圖上前四個節點全自動，不需要人力，維運人員只需要讀報告。

[p6] 成效 (0:50)

安全上，導入前 Web 和 DB 都能從外部存取，很危險；導入後只有 Web 能存取 DB，DB 只在內網。人力上，導入前故障後 1~2 分鐘收到告警信，接著登機器翻 log、查 RDS 指標、查 AWS 操作紀錄、找到根因，共四個人工步驟；導入後告警信之後由 AI 自動調查產出根因報告，只剩一個 AI 自動步驟。
講到「1~2 分鐘告警信」時:
【切分頁 1、2 現場確認】我開場注入的故障，到現在約兩到三分鐘，Alarm 已經觸發，告警信也到了。更完整的證據，在後面的實作。
【若沒到: 「還沒到，後面實作再看」，繼續講】

[p7] 選型 (1/2): 為什麼是 t3.micro (0:45)

EC2 選 t3.micro，看記憶體，不看 CPU。CPU: 每日 50 人次，每次看 5 頁，一天 250 頁；尖峰一小時佔 15%，約 38 頁，約每秒 0.01 頁；WordPress 產生一頁約 0.2 秒 CPU，尖峰需求 0.002 vCPU，t3.micro 穩定提供 0.2 vCPU，只用到 1%。記憶體: 固定開銷約 400MB，加上 6 個程序每個 60MB 共 360MB，合計 760MB。t3.nano 的 0.5GiB 不夠，t3.micro 的 1GiB 才夠。

[p8] 選型 (2/2): 為什麼是 RDS (0:35)

EC2 自建 MySQL 用 t3.micro，每月 9.93 美元，備份、修補、還原要自己處理；RDS 託管 MySQL 用 db.t3.micro、20G，每月 21.74 美元，這些由 AWS 處理。價格出自 AWS Pricing Calculator。每月多約 12 美元，換來省下維運人力，符合我們的痛點。

[p9] 實作 - 架構 (0:55)

Internet 先到 public subnet 的 ALB，掛 ACM 憑證，走 HTTPS 443；後面是 EC2 t3.micro 跑 WordPress；資料庫是 private subnet 的 RDS MySQL，只開 SG 3306，截圖可以看到 Internet access gateway 是 Disabled。
監控流程: EC2 上的 CloudWatch Agent 把 Apache log 推到 CloudWatch，授權靠 EC2 的 IAM role，不是 VPC 網路。Logs 經 Metric Filter 轉成 5xx 指標，Alarm 觸發後平行寄 SNS 告警信，並啟動 investigations 自動調查。

[p10] 故障演練 (1/2): 故障與偵測 (1:10)

剛才開場的注入，就是六步的前三步，現場結果在這裡。
第一步，移除 DB 安全群組的 3306 規則，簡報截圖是終端機輸出，圈起來的 FromPort、ToPort 3306 就是被移除的規則。
【切終端機 2】第二步，網站回傳 500，這是現場剛才打的結果。每個請求約等 60 秒才回，因為封包被丟棄，WordPress 卡到連線逾時才回 500。
【切分頁 1】第三步，5xx 飆高，Alarm 進入 ALARM。
【切分頁 2】這是告警信，現場我在 hh:mm 注入，hh:mm 收到信，約 2 分鐘，也呼應簡報上先前量到的 13:22 到 13:24。
【若現場沒到: 看簡報上的 screenshot6、7，說「先看之前跑的結果」】

[p11] 故障演練 (2/2): 調查與還原 (1:20)

告警信寄出時，investigations 也同時開始自動調查，約需 10 分鐘，所以今天這輪報告還在跑。我先打開之前跑完的結果。
【切分頁 3】亮綠框的 Hypotheses 明確指出 RevokeSecurityGroupIngress 事件，也就是有人移除了安全群組的入站規則，正是我注入的故障。
有了明確根因，我直接 terraform apply。
【切終端機 1】apply 很快，SG rule 被加回。
【切終端機 2，curl 回 200；再重新整理首頁】網站恢復正常。
對照 p6，人工排查從 4 步變成 1 步: 不用登機器翻 log，報告直接指出根因。

[p12] 實作 - 權限安全 (0:40)

讓 AI 查系統，會不會多開一個洞？AI 能查，不能改。investigations role 有三條讀取路徑: Logs、RDS 指標、CloudTrail；指向修改資源的路徑被打叉。截圖是受管政策 AIOpsAssistantPolicy，說明寫 Provides ReadOnly permissions，Action 全是 Get、List、Describe。全部用 IAM role，沒有新增長期金鑰。解決人力問題時，沒有犧牲安全。

[p13] Takeaway + [p14] 結尾 (0:35)

總結，這是安全、省人力的 WordPress 平台，靠內外網隔離加 AI 自動調查。安全上，DB 只在內網，Internet 無法直連；人力上，約 2 分鐘告警，人工排查 4 步變 1 步。今天這輪的 AI 報告再過幾分鐘會完成，Q&A 時我可以現場展示。謝謝大家。

[Q&A 加分] 展示今天這輪報告

【切分頁 4】這是今天現場這輪的調查結果，約 12 分鐘跑完，同樣指出 SG 規則被移除。

七、風險與備援

風險	處理
告警沒在預期時間到	照稿跳過，到 p10 再看；仍沒到就改看簡報 screenshot6、7，說明是之前的結果
curl 卡住或沒回 500	不中斷、不等待，改看簡報 screenshot5；ALB 的健康檢查也會寫入 log，Alarm 可能早於 curl 觸發，這是正常現象
預跑的調查頁面被封存或不見	改用簡報 p11 的 screenshot8，仍照實說明是先前結果
現場念的時間和簡報 13:22 / 13:24 不同	現場念現場時間；簡報上是先前數字，兩邊都是「約 2 分鐘」，不矛盾

八、彩排檢查清單

- 完整彩排兩次以上，一次計時，一次連同現場操作；預跑的調查來不來得及用，彩排後再定。
- 超過時間時，6 分鐘版先砍 p7~p8 的細節，再砍 p9 的監控流程細節，不要加快語速。
- 演練前確認: 網站 200、Alarm 為 `INSUFFICIENT_DATA`、預跑調查頁面仍在。
- Q&A 預期問題: (1) AI 調查誤判怎麼辦，人為什麼還要讀報告？ (2) 為什麼不用 Aurora 或更大的 EC2？ (3) investigations 的成本？【大綱沒有數據，需先查再答】
- 演練結束後，要到 investigations 頁面把調查封存，再 `terraform destroy` (README 清理步驟)；殘留的 investigation group 會讓下次 `terraform apply` 失敗。